

APUNTESWEB · INGENIERÍA INFORMÁTICA

AQRCOMP

Segundo curso · 2.º cuatrimestre · 4 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Sistemas Multinúcleo

1.1 Introducción a la Arquitectura de Computadores y Procesadores Multinúcleo

La arquitectura de computadores es el área de la informática que estudia el diseño y la organización de los procesadores, la memoria y los sistemas de interconexión para optimizar el rendimiento de los sistemas de cómputo.

Dado el crecimiento de la demanda de procesamiento y las limitaciones físicas del modelo tradicional basado en el aumento de frecuencia, los procesadores actuales han evolucionado hacia arquitecturas multinúcleo que explotan el paralelismo.

1.2 Concepto y Tipos de Paralelismo

La **explotación del paralelismo** está relacionada con la **mejora de rendimiento** del sistema. Para poder explotar el paralelismo es necesario modificar las aplicaciones: **el código debe exponer el paralelismo de manera explícita**.

Paralelismo en **arquitecturas**:

- **Instruction-Level Parallelism (ILP)**: se ejecutan a la vez varias instrucciones en distintas fases. Se dice que está **agotado**, a nivel tecnológico ya que no hay muchas mejoras posibles y se deben buscar otros métodos para mejorar el rendimiento.
- **Thread-Level Parallelism (TLP)**: Múltiples hilos ejecutados en paralelo en distintos núcleos.
- **Request-Level Parallelism (RLP)**: Procesamiento de múltiples solicitudes en paralelo.
- **Vector Architectures / GPUs (SIMD)**: Se aplican instrucciones a múltiples datos en paralelo. Para ello agrupan las variables en vectores, realizando muchos cálculos en pocas instrucciones.

1.3 Clasificación de Arquitecturas Paralelas

Existen diferentes criterios que permiten clasificar arquitecturas paralelas: **taxonomía de Flynn**, según la organización del sistema de memoria, según la escalabilidad, disponibilidad de sus componentes, cociente rendimiento/coste, etc.

Es una clasificación de los procesadores basada en el número de flujos de **instrucciones** y **datos** que manejan simultáneamente.

- **SISD**: computador secuencial que puede explotar el **paralelismo a nivel de instrucción**
- **SIMD**: arquitecturas vectoriales, GPUs o extensiones multimedia
- **MISD**: no hay implementaciones comerciales
- **MIMD**: cada procesador tiene sus propias instrucciones y opera sobre sus propios datos.

Categoría	Flujo de Instrucciones	Flujo de Datos	Descripción
SISD (Single Instruction, Single Data)	1	1	Computación secuencial tradicional (Ej: procesadores mononúcleo antiguos).
SIMD (Single Instruction, Multiple Data)	1	Múltiples	Ideal para procesamiento de gráficos y cálculos vectoriales (Ej: GPUs).
MISD (Multiple Instruction, Single Data)	Múltiples	1	Poco común, usado en sistemas tolerantes a fallos.
MIMD (Multiple Instruction, Multiple Data)	Múltiples	Múltiples	Base de los procesadores multinúcleo modernos.

1.4 Conceptos de Procesadores Multinúcleo

1.4.1 Segmentación (pipeline)

La **segmentación** es la ejecución de instrucciones en **etapas**, de manera que cada etapa tiene un **retardo adicional** (para almacenar los resultados de la etapa en registros).

La **profundidad de segmentación** es el número de **etapas de segmentación**.

F₀₄ (*Fan_out of 4*) es una unidad de tiempo equivalente al retardo que tiene una señal al atrevesar un único inversor con 4 inversores conectados a la salida. Su valor exacto depende de la tecnología de fabricación, es decir, del **nodo tecnológico**.

NOTA

Un **nodo tecnológico** se refiere al **tamaño mínimo de los transistores y las interconexiones** en un proceso de fabricación. Este tamaño se mide en nanómetros (nm) y define la generación de un microprocesador.

El **retardo de cada instrucción** en un procesador con segmentación es más alto que sin ella debido a la lógica adicional añadida, pero pasa **menos tiempo entre instrucciones**.

En cada ciclo de reloj se emite una instrucción para ser ejecutada, pero alguna puede quedarse **bloqueada** en alguna etapa (por ejemplo, para buscar un dato en memoria), provocando que las instrucciones posteriores no puedan avanzar, lo cual se conoce como la **burbuja** (parada de emisión), lo que reduce la productividad.

Con segmentación se usa una **frecuencia de reloj más alta** pues el periodo será de la longitud aproximada de una etapa (se intenta que todas tarden aproximadamente lo mismo).

1.4.2 Tecnologías de Fabricación CMOS

Los chips tienen **varias capas de transistores** apiladas para ahorrar superficie. El valor numérico que identifica al **nodo tecnológico** es el ancho **mínimo** de una **conexión de** cobre de **nivel 1**, que esta relacionado directamente con la superficie que ocupa el transistor.

La **ley de Moore** predice que la unidad de transistores por unidad de área se duplica cada 2 años aproximadamente. Cada nodo tecnológico supone una **reducción del 0.7x de las conexiones** entre transistores y del **0.5x del área** de un chip.

En el caso **ideal**, esto supondría que el procesador tiene el **doble de superficie** disponible, pero en **realidad no todas sus partes escalan igual** (especialmente, la memoria DRAM escala mal).

Al principio el área ocupada por la misma cantidad de transistores se **reducía a la mitad** entre nodos tecnológicos, pero **cada vez se reduce menos** (en torno al 0.4x).

1.4.3 Potencia

Potencia Dinámica

La **potencia dinámica** es la energía consumida cuando los transistores cambian de estado (**de 0 a 1 o de 1 a 0**). Esto ocurre cada vez que el procesador ejecuta instrucciones.

$$P_{dinámica} = \frac{1}{2}CV^2f$$

Donde:

- **C** = Carga capacitiva
- **V** = Voltaje de alimentación
- **f** = Frecuencia de reloj

La **carga capacitiva** depende del número de **transistores activos** y de la **tecnología** (que determina la capacitancia de cables y transistores). En CMOS escala ineficientemente, 0.8x de una generación a otra (mientras que transistores x2)

A mayor frecuencia, **más conmutaciones** por segundo → **mayor consumo energético**.

La potencia es **proporcional al cuadrado del voltaje**, lo que significa que **reducir V** ayuda significativamente a disminuir el consumo energético.

Potencia Estática (o de fuga)

La **potencia estática** es la energía consumida **incluso cuando el procesador está inactivo**. Se debe a **corrientes de fuga** que atraviesan los transistores aunque no estén cambiando de estado.

$$P_{estática} = I_{fuga} \times V$$

Donde:

- I_{fuga} = Corriente de fuga
- V = Voltaje de alimentación

Para reducirla:

- **Power gating**: desconectar las partes que no se estén usando.
- Diseños **domain-specific** -> diseños optimizados para una operación muy frecuente.
- Escala con el **número de transistores**, aunque estén sin funcionar. Si se usan cachés SRAM grandes, puede llegar al 50%

Thermal Desing Power (TDP)

El **TDP** caracteriza el consumo de potencia sostenido. Se usa como objetivo en cuanto a **potencia suministrada** y **sistema de refrigeración**. Su valor está entre la potencia pico y la media

La **frecuencia de reloj** se puede **reducir dinámicamente**, es decir, tomar valores distintos en distintas partes del sistema, para limitar el consumo de energía.

Hay un **límite** en las condiciones de **temperatura** en las que puede funcionar un circuito. Se ha llegado a un **límite de disipación** de potencia (100-200W) debido a cuestiones técnicas y económicas.

Mantener el límite en cada nueva generación de microprocesadores requiere **optimizar el consumo de potencia**.

Técnicas para Aumentar la Eficiencia Energética

- **No hacer nada:** Cuando una unidad no está en uso, se **detiene su reloj** para evitar consumo innecesario de energía estática o dinámica.
- **Escalado Dinámico de Voltaje-Frecuencia (DVFS):** Cuando el procesador está inactivo, **reduce su frecuencia y voltaje** para ahorrar energía.
- **Estados de baja potencia en memorias DRAM y discos almacenamiento:** tratando de que las partes inactivas funcionen con frecuencia menor.
- **Overclocking:** trabajar a frecuencia más alta durante breves periodos de tiempo.
 - Puede implicar apagar los núcleos para los que no se sube la frecuencia
 - Limitado por la subida de la temperatura
 - Transparente a la usuario

1.4.4 Ancho de Banda y Latencia

Tiempo de Respuesta, latencia o tiempo real: tiempo entre el inicio y final de una tarea. Incluye todos los sobrecostos del sistema (E/S, acceso a memoria RAM, etc). **Ancho de Banda:** mide la transferencia de información por unidad de tiempo. **El ancho de banda máximo sostenido (BWM)** se diseña para satisfacer cargas de trabajo representativas para el procesador.

$$BWM = \frac{N \times F}{IP \times CPI_{corei}} \quad (\text{bytes/s})$$

Donde:

- **N** = Número de núcleos.
- **F** = Frecuencia del procesador.
- **IP** = Intensidad operacional (instrucciones/byte).
- **CPI** = Ciclos por instrucción.

En la práctica, el *BWM* **no siempre escala entre generaciones:** se suele **duplicar entre nodos tecnológicos** y mantener el mismo diseño de memoria dentro del mismo nodo.

1.5 Métricas de Rendimiento de un Sistema

Se basan en la medida de tiempos de ejecución:

- **Tiempo de CPU:** tiempo de computación en CPU para una tarea concreta.

$$Tiempo_{CPU} = \text{Ciclos de CPU} \times \text{Tiempo de ciclo}$$

$$Tiempo_{CPU} = \frac{\text{Ciclos de CPU}}{\text{Frecuencia de reloj}}$$

$$\text{Ciclos de CPU} = \text{Número de instrucciones} \times \text{CPI}$$

- **Speedup:** aceleración de una version X relativa a una versión Y para un programa en particular

$$\text{SpeedUp} = \frac{T_{original}}{T_{nuevo}}$$

- **CPI (Ciclos por Instrucción):** número medio de ciclos que una instrucción necesita para ejecutarse. Diferentes instrucciones **pueden requerir distinto número de ciclos**.

$$CPI = \frac{\sum(NI_i \times CPI_i)}{N}$$

Hay que tener en cuenta que expresa la media. Por norma general las instrucciones en un MIPS con un pipeline de 5 etapas tienen un $CPI = 1$ pero por diversas causas que se estudiarán en el tema 2 se pueden producir retrasos que lo varíen creando un cierto overhead.

$$CPI_{medio} = CPI + overhead$$

El CPI puede ser < 1 , como veremos en el tema 3.

- **MIPS (Millones de Instrucciones por Segundo)**

$$MIPS = \frac{NI}{T_{cpu} \times 10^6}$$

- **GFLOPS (GigaFLOPS)**

$$GFLOPS = \frac{FLOPS}{T_{cpu} \times 10^9}$$

El gran problema del MIPS Y del GFLOPS es que no son una buena medida de rendimiento, porque que ejecute muchas operaciones por segundo, no implica que programa vaya a tardar menos en ejecutarse que en otro con un MIPS menor.

- **LEY DE AMDAHL:** La posible mejora de rendimiento está limitada por la proporción en que se use la prestación mejorada.

$$t_{mejor} = \frac{t_{\text{parte mejor}}}{\% \text{ de mejora}} + t_{\text{parte no mejor}} \Rightarrow t_{mejor} \geq t_{\text{parte no mejor}}$$

A un código que se ejecutaba en t se le aplica una mejora a un porcentaje de su ejecución F , que lo reduce en un factor M .

$$t' = \frac{F \cdot t}{M} + (1 - F) \cdot t$$

1.6 BenchMarks

Para evaluar el rendimiento de un sistema se puede usar un conjunto específico de programas de prueba conocidos como **BENCHMARKS**.

- La práctica estándar es usar conjuntos de programas de **aplicación real**.
- Los programas de prueba forman una carga con la que el usuario espera **predecir el rendimiento de la carga real del sistema**.

Para indicar las medidas se debe hacer un **informe** de forma que otra persona pueda reproducir los resultados (versión del SO, compilador, entradas, ...). El rendimiento de la máquina medido con un benchmark se suele dar como un **único número**.

El grupo de programas de prueba más popular y completo es el **SPEC** (*Standard Performance Evaluation Corporation*).

- Se usan para medir **tiempo de CPU y productividad**.
- Los 43 programas que se incluyen en la última versión de SPEC para procesador se agrupan en:
 - Carga computacional intensa en **punto flotante** (SPECfp2017).
 - Carga computacional intensa en **enteros** (SPECint2017).
- El **procesador**, la **memoria** y la **E/S** del sistema influyen en el valor medido, junto con el **programa utilizado**.
- Los tiempos de ejecución del sistema deben ser normalizados, para lo que se usa otro sistema como **referencia** (normalmente uno muy antiguo):
 - Ratio SPEC para un programa →

$$SPEC = \frac{T_{CPU\ referencia}}{T_{CPU\ test}}$$

- El test se repite para todos los programas del conjunto SPEC y se computa la **media geométrica** de los resultados: Ratio SPEC para un conjunto de n programas (es hacer una media geométrica de los SPECs)

$$\text{velocidad SPEC} = \sqrt[n]{\prod_{i=1}^n (SPEC_i)}$$

1.7 Paralelismo de Datos

Existen aplicaciones estructuradas en tareas simples denominadas **Kernels** (conjuntos de instrucciones) que operan sobre muchos datos, con potencial de ejecución paralela:

- Por ejemplo, aplicaciones gráficas, multimedia y de realidad virtual
- En ellas es **fácil** disponer de hilos que puedan **operar en paralelo**, lo **difícil** es **proporcionar datos a la velocidad necesaria**.

Esto se logra con **procesadores vectoriales**.

Otra posible solución es incluir en los **núcleos de propósito general** la posibilidad de **procesamiento vectorial**. Actualmente se pueden realizar operaciones en paralelo sobre **vectores de 256 bits** (es decir 8 operandos en float o 4 en double).

Las instrucciones vectoriales aumenta la **complejidad de la programación**. El aumento en velocidad suele compensar. Para tamaños de problema grandes se puede llegar a ganar hasta 8x en tiempo de ejecución si se usan operaciones AVX.

Microprocesadores Basados en Streaming

Los microprocesadores basados en streaming extienden el concepto del procesador vectorial (son usados en las tarjetas gráficas de Nvidia). En lugar de organizar la información en vectores, usan **streams**, que son vectores de estructuras. En lugar de instrucciones vectoriales simples, usan **kernels**, que actúan sobre cada uno de los elementos del stream.

EL **modelo de programación** basado en streams ofrece **más oportunidades de paralelismo** que el de programación paralela.

La **programación** para estas unidades, (se suele realizar CUDA o OpenCL) es en general **más compleja** que en entornos habituales de programa. Ahora se están implementando los **Tensor Cores** especializados en operaciones con matrices.

2. Segmentación

2.1 Introducción a la Segmentación

2.1.1 Ciclo de Instrucción

El **Ciclo de Instrucción** son las etapas necesarias en un caso general para ejecutar cada instrucción.

Etapas de ejecución:

- **IF:** Lectura de instrucción y actualización del registro contador de programa.
- **ID:** decodificación de la instrucción, lectura de registros, extensión de signo de desplazamientos y cálculo de posible dirección de salto.
- **EX:** operación de ALU sobre registros y cálculo de dirección efectiva de salto.
- **MEM:** Lectura o escritura en memoria.
- **WB:** Escritura de resultado en el banco de registros.

2.1.2 Implementación Monociclo

En MIPS, **cada instrucción se ejecuta en un ciclo de reloj** (CPI=1). Entonces, es muy importante decidir una longitud de ciclo de reloj adecuada. Como hay instrucciones con **más etapas** de ejecución que otras, se tendrá que escoger una **longitud de ciclo** que permita que se ejecute la **más lenta**. Como consecuencia, el resto de instrucciones tendrán tiempos muerto durante su ciclo de ejecución.

Opciones para reducir el ciclo de reloj:

- Mejoras **tecnológicas de los circuitos** que permitan reducir el tiempo de ejecución de **cada etapa**.
- Mejoras de la **organización del hardware** que pueda ejecutar más de una instrucción **al mismo tiempo**.

2.1.3 Implementación Multiciclo

La implementación multiciclo y la segmentación son dos métodos de **mejora de la organización del hardware**.

En las **implementaciones multiciclo** cada etapa de ejecución se ejecuta en un ciclo. Para conseguirla hay que **subdividir** la unidad de ejecución y colocar **registros entre etapas**. Si permitimos lanzar a ejecutar una instrucción y mientras se ejecuta ir lanzando la siguiente, la implementación multiciclo se usa como una **unidad segmentada** (pipeline).

La **segmentación** se consigue a partir de una implementación multiciclo si se permite comenzar a ejecutar una instrucción mientras se ejecutan otras. Sin embargo, no todas las instrucciones **terminan al mismo tiempo**. Cuando dos instrucciones necesitan usar la **misma unidad funcional** se pueden producir **conflictos** (hazards). Los **saltos** pueden provocar **paradas**.

El objetivo es aprovechar todas las etapas a la vez para ir progresando en la ejecución de diferentes instrucciones simultáneamente.

- En cada instante habrá como **máximo una instrucción en cada etapa**.
- **Idealmente** se inicia **una instrucción cada ciclo** de reloj.

2.1.4 Tiempo del Pipeline

La complejidad del conjunto de instrucciones afecta directamente a la complejidad del pipeline.

Una unidad convencional de ejecución de N etapas tardará en ejecutar una instrucción la suma del tiempo que tarda cada una de las etapas.

$$T_{comb} = T_1 + \dots + T_N$$

Una unidad **segmentada** con N etapas tardará más en ejecutar cada instrucción por separado, pues tiene que cargar los registros entre cada etapa. El ciclo de reloj debe ser lo suficientemente largo para ejecutar la etapa más lenta y realizar su carga de registros. Por tanto, conviene que **todas las etapas** del pipeline tengan una **duración similar**.

En cada ciclo de reloj se solapa la ejecución de diferentes instrucciones, de manera que, tras una **latencia inicial** igual al tiempo de ejecución de la primera instrucción, se obtiene **una instrucción por ciclo (idealmente)**.

$$T_{clk} = \max(T_R + T_A, \dots, T_R + T_N)$$

2.1.5 Fases del Pipeline

IF-Instruction Fetching

- Envío del PC a memoria
- Lectura de la Instrucción
- Actualización del PC
- En esta etapa se calcula el PC de la siguiente instrucción (útil recordarlo para los saltos que se explican después)

ID-Instruction Decodification

- Decodificación de Instrucciones

- Lectura de Registros
- Extensión de Signo de Desplazamientos
- Calculo de posible dirección de salto

EX- Execution

- Operación de la ALU sobre registros
- Alternativamente, cálculo de la dirección de salto final

MEM- Memoria

- Lectura o escritura en memoria

WB-WriteBack

- Escritura del resultado en banco de registros

2.2 Riesgos en la Ejecución (Hazards)

Un riesgo es una situación que **impide que la siguiente instrucción pueda comenzar en el ciclo de reloj previsto**. Estas situaciones reducen el rendimiento de las arquitecturas segmentadas. Los riesgos pueden ser de tres tipos: estructurales, de datos o de control. La aproximación más simple (y menos eficiente) consiste en **detener el flujo de instrucciones** hasta que se elimina el riesgo.

El **CPI ideal** de un procesador segmentado es 1. Los ciclos del pipeline en los que no se computa se denominan **burbujas del pipeline** y en ellos se introduce en el pipeline un **código de no operación** (NOP).

2.2.1 Riesgos Estructurales

Los riesgos estructurales se producen cuando el **hardware** no puede soportar todas las posibles **secuencias de instrucciones**. Esto se produce si dos etapas necesitan hacer **uso del mismo recurso hardware**.

Las razones suelen ser la presencia de unidades funcionales que no están totalmente segmentadas o unidades funcionales no duplicadas. En general, los riesgos estructurales se pueden evitar en el diseño pero encarecen el hardware resultante.

2.2.2 Riesgos de Datos

Un riesgo de datos se produce cuando la segmentación **modifica el orden de acceso de lectura/escritura a los operandos**. Los riesgos de datos pueden ser de tres tipos:

- RAW (Read After Write)
- WAR (Write After Read)
- WAW (Write After Write).

De estos tres, solamente los riesgos **RAW** pueden darse en una arquitectura de cinco etapas tipo MIPS. Los riesgos de tipo RAW pueden resolverse en algunos casos mediante el uso de la técnica del **envío adelantado (forwarding)**.

RAW - Read After Write

Una instrucción intenta leer un dato justo antes de que otra anterior lo escriba. Las instrucciones no pueden ejecutarse en paralelo ni solaparse completamente, provoca una **parada**. Se pueden detectar por **hardware** o por **compilador**

WAR - Write After Read

Una instrucción intenta modificar un dato antes de que otra anterior lo lea. No puede ocurrir en un MIPS con pipeline de 5 etapas ya que ID es la etapa 2 y WB es la 5, **no provoca una parada**.

WAW - Write After Write

Una instrucción intenta escribir un dato antes de que otra anterior lo escriba. No puede ocurrir en un MIPS con pipeline de 5 etapas, pues estas siempre se ejecutan en el mismo orden, **no provoca una parada**.

Soluciones

WAR o WAW: renombrado de registros.

- Renombrado **estático:** por el compilador
- Renombrado **dinámico:** por el hardware

RAW:

- **Bloquear** la instrucción hasta que se realice la escritura necesaria.
- El compilador **reordena el código** para mitigar el riesgo, introduce una secuencia de instrucciones independientes entre las conflictivas. Si no hay instrucciones independientes inserta instrucciones **NOP**.
- **Envío adelantado** (forwarding, bypassing, anticipación)

Fowarding

El **fowarding** consiste en enviar resultados de las últimas etapas del pipeline a las primeras para evitar esperas. Por tanto requiere **conexiones adicionales**.

No hace falta esperar a que el resultado se escriba en el banco de registros. Ya están en los registros de segmentación. Se puede usar ese valor en vez del que hay en el banco de registros.

Los resultados de las fases **EX** y **MEM** se escriben en **registros de entrada a ALU**. La lógica de forwarding selecciona entre entradas reales y registros de forwarding.

No todos los riesgos se pueden evitar con forwarding. Si el riesgo no se puede evitar se debe introducir una **detención**.

El control de interbloqueo de instrucciones es un proceso que consiste en detectar cuándo una instrucción **no puede avanzar por dependencia** en operandos con otra, **aunque se aplique adelantamiento**, provocando que se **pare la emisión de instrucciones** hasta que se solucione el bloqueo. La **duración** de la detección depende del **tipo** de instrucciones involucradas. Se detecta en la etapa **ID** pues en ella se averigua qué registros son operandos de entrada. Se activa una parada si:

- En la etapa ID hay una instrucción tipo ALU, salto o de almacenamiento.
- En la etapa EX se está ejecutando una carga
- El registro destino de la carga es operando de entrada de la instrucción que está en etapa ID

2.2.3 Riesgos de Control

Un riesgo de control se produce en una instrucción de bifurcación cuando no dispone todavía del **valor sobre el que se debe tomar la decisión de salto**. Los riesgos de control pueden resolverse en **tiempo de compilación** (soluciones estáticas) o en **tiempo de ejecución** (soluciones dinámicas).

Las soluciones estáticas ante los riesgos de control pueden ir desde la congelación del pipeline o la predicción prefijada (predecir siempre a no tomado o siempre a tomado), hasta el uso de bifurcaciones con ranuras de retraso.

Las soluciones dinámicas pueden usar una tabla histórica de saltos (BHT - Branch History Table) y una máquina de estados para realizar la predicción. De esta manera se tiene una máquina de estados asociada a cada entrada de la tabla.

Llamaremos **salto tomado** a la situación en la que se modifica el PC y **salto no tomado** a aquella en la que no se modifica.

Soluciones Estáticas

Congelar el Pipeline

Si la instrucción actual es un salto → parar o eliminar del pipeline instrucciones posteriores hasta que se conozca el destino. El **destino** de la bifurcación se conoce en la etapa ID e implica repetir el **FETCH** de la siguiente instrucción. Esta repetición equivale a una detención.

Predicción Prefijada

No tomada

Asumir que el salto no será tomado. Se evita modificar el estado del **procesador** hasta que se tiene la confirmación de que el salto no se toma. Si el salto se toma, las instrucciones siguientes se retiran del pipeline y se capta la instrucción en el destino del salto. Transformar instrucciones en NOP.

Tomada

Asumir que el salto será tomado. Tan pronto como se **decodifica** el salto y se calcula el destino se comienza a **captar instrucciones del destino**. En pipeline de 5 etapas no aporta ventajas. No se conoce dirección destino antes que decisión de salto. Útil en procesadores con condiciones complejas y lentas.

Decisión Retardada

La bifurcación se produce después de ejecutar las n instrucciones posteriores a la propia instrucción de bifurcación. En pipeline de 5 etapas → **1 ranura de retraso** (delay slot).

Las instrucciones $I1, I2, \dots, IN$ se ejecutan independientemente del sentido de la condición de salto. La instrucción $IN + 1$ solamente se ejecuta si no se produce el salto.

Caso de salto retrasada con una ranura de retraso. Se espera siempre una instrucción antes de tomar el salto. Es responsabilidad del programador poner código útil en la ranura.

Soluciones Dinámicas

ROBANDO UN POCO DE INFO PARA VARIAR

<https://www.youtube.com/watch?v=L7NfhA23Fjg>

Las soluciones dinámicas pueden usar una tabla histórica de saltos (BHT - Branch History Table) y una máquina de estados para realizar la predicción. De esta manera se tiene una máquina de estados

asociada a cada entrada de la tabla. Son muy útiles cuando usamos pipelines muy largos pero implican un alto coste hardware.

Predictor de 1 BIT

La idea es que si un salto fue efectivo, lo más probable es que vuelva a serlo en un futuro. Para aprovechar esto necesitamos un buffer de predicción de salto. Este buffer es una tabla de un único bit para anotar si el salto fue **tomado** o **no tomado**. A este buffer se accede mirando los lsb de la dirección del salto, y así sabemos si fue tomado o no. En caso de predicción errónea se invierten los bits.

Si tenemos un bucle interno y otro externo, si el salto se toma siempre en el interno, por cada vez que itere el externo, va a volver a tener ese fallo el interno.

Predictor de 2 BITS

La idea es darle una segunda oportunidad antes de cambiar la predicción (inercia). BTH de 2 bits: cuatro estados posibles. Así le damos la oportunidad de reconsiderar sus errores y disminuir el número de fallos. Se puede generalizar para n bits con 2^n estados posibles, aunque son poco adaptativos y por eso no se suelen usar.

Predictor Basado en Información Local Tiene en cuenta la historia del comportamiento **de un mismo salto** para tomar la decisión. - Se almacena el resultado de cada salto correspondiente a sus últimas k ocurrencias - Con estos bits se accede a los bits de estado que informan de la predicción - Una vez conocido el resultado real del salto, se actualiza el contenido de la memoria de historia y el filtro de decisión - Una vez conocido el resultado real del salto, se actualiza el contenido de la memoria de historia y el filtro de decisión.

Usa **dos filtros de decisión** y así evita el 100% de los fallos que sucederían al usar un predictor de 1 bit para secuencias tipo T NT T NT...

Predictor Basado en Información Global

Tiene en cuenta la historia del comportamiento de **las últimas N instrucciones de salto** para tomar una decisión. Se usa un **registro de desplazamiento** para almacenar el **resultado de los saltos**. El contenido del registro indexa una **memoria** que guarda los **bits de estado** correspondientes al filtro de decisión.

Predictor Combinado

Combina un predictor **local** y otro **global**. La **tasa de acierto** de los saltos en procesadores reales depende del **tipo de programa** y suele estar entre 85% y 99%

2.3 Operaciones Multiciclo.

La asignación de un único ciclo a las operaciones de coma flotante requiere un ciclo de reloj extremadamente largo o el uso de una lógica de coma flotante muy compleja (con el consiguiente consumo de recursos). La alternativa a estas opciones es la segmentación de la unidad de coma flotante, por lo que estas instrucciones requerirán múltiples ciclos en la etapa de ejecución.

Las instrucciones en punto flotante tienen la **misma segmentación** que las enteras, pero con las siguientes modificaciones:

- La etapa EX tiene **diferente latencia** dependiendo de la **instrucción**.
- Existen **diversas unidades** funcionales para cada **tipo de operación**.

Se añade un **banco de registros** separado para operaciones en **punto flotante**. Se añade una **unidad de multiplicación** segmentada de **7 etapas** en **enteros y punto flotante**. Se añade una **unidad de suma de 4 etapas** en punto flotante. Se añade una **unidad de división no segmentada** que requiere **varios ciclos** reutilizando la **misma etapa** (en esta unidad no se puede ejecutar una instrucción por ciclo).

Chequeo de riesgos estructurales: hay que detectarlos en la unidad de DIV y en la etapa de WB. Para detectarlos:

- Para el paso de la instrucción a EX si es una división y hay otra división en ejecución.
- Para la emisión si se detecta que van a llegar al WB más instrucciones que puertas de acceso de escritura al banco de registros.

Chequeo de **riesgos por dependencias RAW:** similar al caso de pipeline simple con una única etapa de ejecución pero con más casos posibles.

Chequeo de **riesgos por dependencias WAW:** son **poco comunes**. Para detectarlos:

- Determinar si hay una instrucción en una etapa posterior a ID que tiene como destino el mismo registro que la etapa de ejecución, pero que haría la escritura del registro un número de ciclos posterior.
- Si sucede, para el paso a EX de la instrucción que está en una etapa posterior a ID.

2.4 Excepciones

Las **excepciones** se producen por diferentes **eventos en la CPU** (código de operación indefinido, overflows, syscalls,...) y requieren que el programa se ejecute de nuevo **desde el punto en que se produjo la excepción**.

Para poder restablecer el estado es necesario que se haya **guardado** el contenido de los **registros** y de la **memoria**.

- Las excepciones que permiten recuperar un estado anterior por completo se llaman **excepciones precisas**.
- Es complicado gestionarlas sin sacrificar el rendimiento, ya que reestablecer el estado del sistema significa almacenar información y perder ciclos.

Posibles soluciones:

- Incorporar un **banco de registros adicional** para ayudar a guardar los valores producidos por **cada instrucción**.
- Utilizar una **rutina software** que permita **repetir la ejecución** de instrucciones desde la que produjo la excepción.
- **Parar la emisión de instrucciones** si en la capa EX se detecta **riesgo de excepción**.

Si suceden múltiples excepciones anidadas, se gestiona **la más antigua primera**.

2.5 MIPS R4000 (Ha caído en examen)

3. Paralelismo a Nivel de Instrucción

3.1 Instruction Level Parallelism

El paralelismo de instrucción es aplicable dentro de cada bloque básico (secuencia de instrucciones sin saltos). Sin embargo, la longitud media de un bloque básico es de 3 a 6 instrucciones lo que reduce bastante su posible aprovechamiento. Una técnica para mejorar el aprovechamiento del ILP dentro de un bucle se conoce como **desenrollamiento de bucles**. También podemos entrelazar ejecución de instrucciones no relacionadas y rellenar detenciones con instrucciones.

El **desenrollamiento** de bucles consiste en entrelazar la ejecución de instrucciones de varias iteraciones de un bucle. Al tratarse de instrucciones **no relacionadas** no generan dependencias y **permiten un mejor aprovechamiento de la arquitectura segmentada**.

(Cuando en los enunciados pone planificación de lazo se refiere a que reordenemos o desenrollemos el código que nos dan).

3.2 Emisión Múltiple

La **segmentación** permite ejecutar varias instrucciones a la vez. El objetivo de explotar el **ILP** (Instruction Level Parallelism) es **optimizar el CPI**:

$$CPI = CPI_{ideal} + \text{Paradas por riesgos estructurales} + \text{Paradas por riesgos de control} + P$$

La predicción de saltos realizada por el compilador, el unrolling, etc. ayudan a exponer más el paralelismo.

Hay dos alternativas para **mejorar el ILP**:

- Hacer más **profundo el pipeline** (tema2) -> menos trabajo por etapa -> T_{clock} más corto -> mayor f_{clock}
- **Emisión múltiple** (tema3) -> se replican las etapas del pipeline en **múltiples pipelines** en los que se comenzarán **varias instrucciones en cada ciclo**. Entonces $CPI < 1$, por lo que en su lugar se usa el **IPC** (numero de instrucciones por ciclo) como medida de rapidez.

3.3 Emisión Múltiple Estática

En la **emisión múltiple estática** el **compilador** agrupa instrucciones en **paquetes de emisión** para emitirlas juntas. Un **paquete de emisión** es un grupo de instrucciones que pueden ser **emitidas** en un solo

ciclo, y viene determinado por los **recursos del pipeline**. Usan **planificación estática** donde el **compilador** debe **eliminar algunos/todos los riesgos**:

- En un paquete de emisión sólo puede haber instrucciones **sin dependencias entre ellas** (si no, no se podrían ejecutar simultáneamente).
- Posiblemente haya **dependencias entre paquetes** -> su tratamiento varía entre ISAs. **El compilador debe saberlo**.
- Se rellenará el resto del paquete con **NOP** si no se encuentran las instrucciones adecuadas.

Para poder hacer la emisión dual debemos añadir hardware adicional:

Las arquitecturas **VLIW** (Very Long Instruction Word) ven los paquetes como una **instrucción muy larga** por ejemplo:

- Una operación con enteros o una operación de salto
- Dos operaciones independientes en punto flotante
- Dos referencias a memoria independientes

Es necesario encontrar paralelismo estáticamente ya que no hay hardware de detección de riesgos. Gran tamaño del código. Esto implica **hardware** de la CPU más **simplificado**, menor **consumo de potencia**.

Se forman paquetes de emisión con una instrucción **ALU** o un **salto** seguida de una instrucción de **load** o **store**. Las instrucciones no utilizadas se rellenan con **NOP**.

3.4 Emisión Múltiple Dinámica La **emisión múltiple estática** la **cpu** examina el flujo de instrucciones y selecciona las que se deben emitir en cada ciclo. Los procesadores que la realizan denominan **procesadores superescalares**.

Evita la necesidad de **planificación del compilador**, ya que se realiza por hardware, por lo que tampoco debe conocer como es el hardware. La CPU asegura la semántica del código, pero da lugar a CPUs más complicadas en cuanto a hardware. Permite a la CPU emitir varias instrucciones que:

- Se **ejecutan fuera de orden** (para evitar paradas).
- **Terminan fuera de orden** en muchos casos.
- Pero el **WB** respeta la semántica del programa

La **planificación dinámica** consiste en **reordenar** las instrucciones para reducir las paradas a la vez que se mantiene el flujo de datos del programa. Puede gestionar casos en los que las **dependencias** **no** se conocen en **tiempo de compilación**.

Motivos para hacer planificación dinámica:

- No todas las paradas son **predecible** (por ejemplo, los fallos de caché).

- No siempre se pueden planificar estáticamente, por ejemplo, si hay instrucciones de **salto**, ya que el **resultado** del salto se determina **dinámicamente**.
- **Diferentes implementaciones de un ISA** pueden tener diferentes **latencias** y dar lugar a diferentes **riesgos**.
- **Instruction commit**: proceso que se produce cuando una instrucción actualiza su resultado en el **archivo de registros**.

Tenemos el siguiente **camino de datos**:

- **Lectura de múltiples instrucciones en orden**
- **Decodificación de múltiples instrucciones en orden**
- **Distribución/Emisión de multiples instrucciones en orden**: Cada instrucción se distribuye en una **estación de reserva**, y se almacena también en el buffer de terminación.
 - Se realiza el **renombramiento de registros** para resolver dependencias **WAR y WAW**.
 - Si los operandos fuente de la instrucción están disponibles en el **banco de registros** o en el **buffer de terminación/reodenamiento**, se copian en la **estación de reserva**, y se realiza la **emisión de la instrucción** (envío a una unidad de ejecución) en cuanto la unidad está disponible.
 - Si falta algún operando fuente en la instrucción (por una dependencia), la instrucción queda retenida en la estación de reserva hasta que el dato está disponible; cuando la unidad funcional correspondiente produce el dato esperado, se copia en la estación de reserva y la instrucción queda lista para su ejecución. El renombrado de registros se realiza mediante **estaciones de reserva (RS)**, que tienen una **entrada** por cada **instrucción en ejecución**.

Campos de una entrada en la RS:

- *B*: indica si la entrada está ocupada por una instrucción o libre
 - *INST*: campo de código de instrucción.
 - *OP1* y *OP2*: operandos de entrada de la instrucción: proceden de registros o de caminos de adelantamiento de las unidades de ejecución. Pueden ser un identificador de registro o de renombrado, es decir, que su valor está siendo calculado.
 - *V1* y *V2*: indica si los operandos *OP1* y *OP2* son válidos.
 - *R*: indica si la instrucción está lista para la emisión.
- **Ejecución** de la instrucción en la unidad funcional: al terminar, se escribe el resultado en el **buffer de terminación** (y en las estaciones de reserva que están a la espera)
 - **Terminación**: a medida que los resultados se copian en el buffer de terminación, se escriben en el banco de registros **en orden** y las instrucciones se terminan

- **Retirada:** las instrucciones de almacenamiento realizan la escritura en memoria

Algoritmo de Tomasulo

Algoritmo diseñado para permitir a un procesador ejecutar instrucciones fuera de orden, este algoritmo permite el lanzamiento de las instrucciones de WAR y WAW sin detener la ejecución, del mismo modo utiliza un bus de datos común (CDB) en el que los valores calculados son enviados a todas las estaciones de reserva que lo necesiten **Emisión:**

- Se lee la instrucción del buffer de instrucciones.
- Se reserva una estación si está disponible.
- Se renombra el destino si es necesario.
- Si los operandos están listos, se colocan. Si no, se apunta al productor (quién lo generará).

Ejecución:

- Una vez que todos los operandos están listos, la instrucción se ejecuta.
- La ejecución puede tardar varios ciclos.

Escribir resultado:

- El resultado se envía por el **CDB**.
- Todas las unidades que esperaban ese resultado lo reciben.
- El registro de destino se actualiza si aún corresponde a esa instrucción.

Dependencias:

- **RAW (Read After Write):** Tomasulo espera hasta que el dato esté disponible.
- **WAW (Write After Write):** Se evita con renombramiento de registros.
- **WAR (Write After Read):** Se evita porque las instrucciones no escriben directamente hasta la fase final.

MANDA HUEVOS IR HASTA ALEMANIA PARA TENER UNA EXPLICACIÓN DECENTE

<https://www.youtube.com/watch?v=YH2fFu-35L8>

3.4 Especulación Basada en Hardware

La **especulación** consiste en hacer conjeturas sobre de lo que hará una instrucción.

- Comenzar la operación tan pronto como sea posible
- Comprobar si se acertó:
 - Si se acertó: completar la operación

- Si no se acertó: retroceder y corregir

Se realiza tanto en la emisión **multiple** como en dinámica.

Tipos de especulación:

- Especulación realizada por el **compilador**: reordena instrucciones (ej, mover la carga antes de un salto). Si la especulación es incorrecta, puede incluir **instrucciones de arreglo** para recuperarse.
- Especulación realizada por **hardware**: busca por adelantado instrucciones para ejecutar. Mete los resultados de ejecutar la instrucción en un buffer hasta que se determine si se necesitan realmente, si la especulación es incorrecta se **vacían los buffers**.

Especulación acerca del Salto

Consiste en especular acerca del resultado de un salto y continuar emitiendo instrucciones asumiendo que son las correctas. No se actualizan los registros (Instruction Commit) hasta que se determine el resultado de salto.

Especulación acerca de la Carga

Trata de evitar el retardo por carga y fallo caché.

- Predecir la dirección efectiva
- Predecir el valor cargado
- Pasar los valores almacenado a la unidad de carga No se actualizan los registros hasta que se resuelve la especulación.

Buffers de Reordenamiento (ROB)

Posibles estados para una instrucción:

- **En ejecución**: la instrucción no acabó de ejecutarse
- **Finalizada**: la ejecución acabó y se escribe el resultado en un **registro renombrado** o en un **buffer** previo a escribir memoria (si es un store)
- **Completada**: modifica los registros del núcleo.
- **Retirada**:
 - Si el destino es un registro: coincide con **completada**
 - Si el destino es la memoria: se escribe en la caché de datos

4. Subsistema de Memoria Compartida en Procesadores Multinúcleo (Coherencia caché)

<https://ocw.uc3m.es/mod/page/view.php?id=2822>

4.1 Introducción

En las memorias caché hay dos **tipos de datos**:

- Datos **privados**: datos usados por un único procesador
- Datos **compartidos**: datos usados por varios procesadores. Estos datos pueden **replicarse** en varias cachés, lo que reduce la **contención en el acceso a memoria**, pues cada procesador accede a su copia local.

Coherencia: define el **comportamiento** de **lecturas** y **escrituras** en la misma posición de memoria y, por tanto, que todos los procesadores en un sistema multiprocesador tengan **la misma visión** de memoria

Consistencia: define el **orden** en que se ejecutan las operaciones de memoria unas respecto a otras y, por tanto, que esas operaciones se realicen en el orden del **programa secuencial**.

- En los **sistemas monoprocesador** el orden en que se actualicen las posiciones de memoria afecta a la memoria principal y a la caché
- En los **sistemas multiprocesador** el orden en que se actualicen las posiciones de memoria afecta a la memoria principal y a la caché de todos los procesadores.

4.1.1 Multiprocesadores UMA

En los multiprocesadores UMA (*Uniform Memory Access* o multiprocesadores simétricos **SMP**) todos los procesadores comparten una **única memoria** con la **misma latencia** para todos ellos. Tienen un número de cores pequeño (≤ 8)

Condiciones para que haya coherencia

Un sistema de memoria es **coherente** si cualquier lectura de una dirección devuelve el valor más reciente que se haya escrito para esa dirección

- **Preservar el orden del programa**: si un procesador P escribe la posición X y luego la lee, el valor devuelto es siempre el valor escrito por P . Se cumple si no hay escrituras por parte de otro procesador entre la escritura y lectura de P

- **Vista coherente de la memoria:** si un procesador Q escribe la posición X y luego otro procesador P la lee, leerá el valor escrito por Q . Se cumple si la escritura y lectura están suficientemente separadas y entre los dos accesos no hay ninguna escritura de X
- **Serialización de escrituras:** dos escrituras de la misma posición por dos procesadores cualesquiera son vistas en ese mismo orden por todos los procesadores.

4.1.2 Métodos de Actualización de Memoria Principal

- **Escritura directa (ED o write_through):** siempre que se modifica una dirección en la caché, se modifica la memoria principal.
 - Cada escritura supone **utilización de la red**, por lo que es poco eficiente en sistemas multiprocesador, en los que varios procesos pueden escribir a la vez.
 - Se produce **incoherencia entre cachés**, pero no entre caché y memoria
- **Post-escritura (PE o write back):** cuando un procesador modifica una dirección de memoria sólo se escribe en la caché.
 - El dato no se transfiere a memoria principal hasta que se eliminan de la caché, por lo que se puede **escribir** varias veces en un bloque de caché **sin acceder a memoria** principal.
 - Se produce **incoherencia entre cachés**, y **también entre caché y memoria**.

Entonces, se producirán falta de coherencia tanto usando ED como PE. Posibles soluciones:

- **Hardware específico.**
- Otras soluciones: declaración de zonas de memoria **no cacheables** o **con escritura inmediata** (para dispositivos de salida)
- Solución estándar: **protocolos de coherencia caché**

4.2 Protocolos de Coherencia Caché

Los **protocolos de coherencia caché** hacen que cada **escritura** sea **visible** a todos los procesadores y **propagan** de forma fiable un nuevo **valor** escrito. Se basan en seguir la pista al **estado** de un bloque (línea) de datos caché compartido.

4.2.1 Métodos de Propagación de Escrituras a las Cachés

- **Escritura con actualización (WU o write-update):**
 - Si el dato es **compartido**: lo actualiza en todas las cachés que lo contengan (actúa como caché **ED**).
 - Si el dato es **privado**: no es necesario actualizar en otras cachés (actúa como caché **PE**).

- **Escritura con Invalidación** (WI o write-invalidate): cuando se realiza una escritura, se distribuye una **señal de invalidación** a través del bus y todas las cachés comprobarán si tienen una copia de ese dato. Las que lo tengan, invalidarán la línea que lo contiene.
 - Es la solución más **común**
 - Se usa en los **protocolos de snooping**

Diferencias

- Múltiples escrituras a la **misma palabra** (sin lecturas por el medio):
 - WU: una difusión de escritura cada vez
 - WI: solo la primera escritura a la palabra genera una invalidación
- Múltiples escrituras a la **misma línea** (en distintas palabras):
 - WU: una difusión de escritura por cada palabra
 - WI: solo la primera escritura a cualquier palabra genera una invalidación
- **Retardo** entre escribir una palabra en un procesador y leer el valor escrito en otro:
 - WU: es más rápido ya que el dato es actualizado inmediatamente en la caché del lector
 - WI: es más lento ya que el lector es invalidado y no puede leer el dato hasta que reciba una copia actualizada

4.2.2 Tipos de Protocolos

- **Snooping o espionaje:** cada caché tiene información sobre el estado de compartición de los bloques que contiene.
 - Típico de los sistemas de **memoria compartida** con un **bus común**: los controladores caché espían el bus para determinar si tienen o no una copia del bloque compartido solicitado
 - El **tamaño** de la información de coherencia es proporcional al número de **bloques de la caché**
 - No escalan bien con el **número de cachés** del sistema porque generan mucho **tráfico** en el bus
- **Basados en directorios:** el estado de un bloque de memoria física se mantiene en una única localización, el directorio.
 - Típico de sistemas **escalables**: no necesitan que todas las cachés estén comunicadas por un bus común, sólo tiene que haber comunicaciones entre las cachés implicadas
 - Información que contiene el directorio:
 - Cachés que tienen copia del bloque
 - Si ha sido modificado o no
 - Por tanto, **el tamaño es proporcional al número de líneas de la MP**

- Es una solución **centralizada**: el estado de compartición de un bloque está siempre en la misma posición, por lo que el directorio es un **cuello de botella**. Las entradas del directorio pueden estar **distribuidas** en distintas memorias, lo cual mejora este problema, pero **reduce la contención de memoria**

4.3 Protocolos de Snooping

La **invalidación** de escrituras es una estrategia que garantiza que un procesador tiene **acceso exclusivo** a un bloque **antes** de realizar una **escritura**. Uso del bus de memoria para invalidación:

- El procesador adquiere el bus y difunde la dirección a invalidar
- Los demás procesadores espían el bus
- Cada procesador comprueba si está la dirección en su caché. Si es así, invalida el bloque que la contiene con el **bit de validez**. Así, **no puede haber dos escrituras simultáneas**, pues el uso exclusivo del bus serializa las escrituras.

4.3.1 MSI, Protocolo de Invalidación de 3 Estados

El protocolo MSI está basado en un **sistema secuencial síncrono** en el que cada bloque de caché tiene un estado que cambia en función de **peticiones** del **procesador** y **peticiones del bus**.

Posibles Estados de Cada Bloque de Caché

- **Modificado (M)**: este es el **único componente** que tiene una **copia válida** del bloque, el resto de cachés y la **memoria** tienen una copia **no actualizada**.
 - La caché debe **proporcionar** el bloque si espía en el bus que algún componente lo solicita
 - La caché debe **invalidar** el bloque si otra solicita una copia para modificarlo
- **Compartido (S shared)**: este bloque está presente en **otras ubicaciones** y **todas** las copias están **actualizadas**.
 - La caché debe **invalidar** el bloque si otra lo modifica
- **Inválido (I)**: el bloque **no está físicamente** en la caché o su valor ha sido invalidado por haberse producido una **escritura** en **otra caché**

Peticiones que puede generar un nodo con caché

- Petición de **lectura de un bloque (PtLec)**: se genera como consecuencia de la **lectura del procesador (PrLec)** de una dirección que **no se encuentra en la caché**.
 1. El controlador de la caché pone en el bus la dirección que se desea acceder
 2. El sistema proporcionará el bloque donde se encuentra la dirección solicitada

- Petición de **acceso exclusivo a un bloque (PtLecEx)**: se genera como consecuencia de la **escritura del procesador (PrEsc)** en una dirección cuyo bloque en la caché se encuentra en estado **S** o **I** (incluida la ausencia del bloque).
 1. El controlador pone en el bus la dirección en la que se quiere escribir
 2. El resto de las cachés invalidan sus copias
 3. La memoria invalida su copia
- Petición de **postescritura (PtEsc)**: se genera cuando el controlador de caché **reemplaza** un bloque **M** como consecuencia del acceso a un bloque que no se encuentra en la caché
 1. El controlador de la caché pone en el bus la dirección del bloque a escribir en memoria y su contenido
- Paquete de **reconocimiento** con el bloque solicitado por una caché (**RpBloque**): se genera cuando un nodo **observa** en el bus una **petición** de un bloque (que incluye lectura) si dicho nodo está en su caché en estado **M** (su copia es la única válida en todo el sistema).

Problemas asociados a la Invalidación

- **Falsa compartición (false sharing)**:
 - La falsa compartición se produce cuando dos variables sin relación entre ellas se encuentran dentro del mismo bloque caché (línea).
 - Todo el bloque se transmite entre procesadores, aunque se esté accediendo a diferentes variables.
 - Como consecuencia se producen fallos caché de falsa compartición: un procesador escribe en una línea compartida y la invalida, luego otro procesador lee una palabra diferente de la línea.
- Fallos de **compartición verdadera (true sharing)**: Un procesador escribe en bloque compartido e invalida y luego otro procesador lee del bloque compartido.

z

4.3.2 MESI, Protocolo de Invalidación de 4 Estados

En MSI, al escribir a una línea en estado S, se debe invalidar en otras cachés y pasar a M, y podría requerir una escritura a memoria si el protocolo la necesita.

En MESI, si está en estado E, el procesador puede escribir directamente y cambiar a M sin necesidad de comunicarse con otros procesadores ni escribir en memoria.

Reduce tráfico de bus:

- El estado E indica que no hay necesidad de notificar a otras cachés.
- Esto **reduce las transacciones en el bus** y mejora el rendimiento.

Por eso, en el protocolo MESI, se define el nuevo estado **Exclusivo**.

Posibles estados de cada Bloque Cache

- **Modificado(M):** esta es la **única caché** que tiene una **copia válida** del bloque y al **memoria** tiene una copia **no actualizada**
 - La caché debe **proporcionar** el bloque si espía en el bus que algún componente lo solicita.
 - La caché debe **invalidar** el bloque si otra solicita una copia exclusiva para modificarlo
- **Exclusivo (E):** esta es la **única caché** que tiene una **copia válida** del bloque y la **memoria** tiene una copia **que sí que está actualizada**.
 - La caché debe **invalidar** el bloque si otra solicita una copia para modificarlo
- **Compartido (S shared):** este bloque es válido en **esta caché**, en **al menos otra caché** y en **memoria**.
 - La caché debe **invalidar** el bloque si otra solicita una copia exclusiva para modificarlo
- **Inválido (I):** el bloque **no está físicamente** en la caché o su valor ha sido invalidado por haberse producido una **escritura** en **otra caché**.

4.3.4 Problemas de Snooping

El **bus** de memoria compartida y el **ancho de banda** que requieren los protocolos de snooping es un **cuello de botella** para **escalar** los SMPs. Posibles soluciones:

- Usar **redes de interconexión punto a punto** con memoria en **bancos**-> tienen mayor ancho de banda
- Usar un **protocolo basado en directorio**

4.3.4 Intento de Explicación del Nuevo Sistema de Caché que Dora le copió a la UCM y que no explicó en clase (no fui a clase).

Se puede medio intuir que dada la tabla esta, no podemos distinguir con visto en fucomp acerca de la división de la instrucción para saber en que línea colocar los datos. Pero dada la imagen se puede razonar tal y como puse yo.

Otro ejemplo:

Si nos dice que es **totalmente asociativa** supongo que se puede sustituir por cualquier línea, a no ser que especifique algún tipo de algoritmo.

4.4 Protocolos Basados en Directorios

Los **protocolos basados en directorios** almacenan la información sobre el estado de compartición de cada bloque de memoria en un directorio. Reducen el tráfico a través de la red ya que se hace **envío selectivo de ordenes**. Se usan:

- Si la **difusión es costosa**
- Si se necesita mayor **escalabilidad**
- **Multiprocesadores** con una **red escalable** (como los de la foto anterior). El directorio local sólo almacena información de los bloques de la memoria local.
- **Multicores** con **caché externa compartida**. (L3)

Cada entrada del directorio informa de:

- Qué caches tienen copia del bloque
- Bits de estado del bloque en cada caché

Transferencias Generadas

- En los protocolos de **snooping**: las transiciones de un bloque entre estados suponen **difusiones**
- En los protocolos **basados en directorio**: las transiciones de un bloque entre estados suponen **transferencias punto a punto**. Las transferencias las genera el controlador de coherencia caché de los nodos implicados

4.4.1 Estados de un Bloque en Caché y Directorio

- Cachés: consideramos una implementación MESI
- Directorio:
 - Estado **local**: el bloque está actualizado en la memoria y no hay copias en ninguna caché
 - Estado **compartido**: el bloque está actualizado en memoria y hay varias copias válidas del bloque en cachés. EL directorio informa de qué caches tienen distintas copias válidas
 - Estado **exclusivo**: el bloque puede no estar actualizado en memoria y hay una copia en una caché en M o E. El directorio informa de qué caché tiene la copia
 - Algunos estados de transición (pendientes de algún paquete para estabilizar su estado, ...).

4.4.2 Clasificación del Directorio

- Directorio **centralizado**: una entrada para cada bloque de memoria con información de estado y de caches con copia. El directorio atiende a todas las peticiones, por lo que es un **cuello de botella**.
- Directorio **distribuido entre módulos de MP**: cada módulo tiene un subdirectorio con la información relativa a sus bloques de memoria. Los subdirectorios atienden en **paralelo** las peticiones de acceso.

- Directorio de **vector de bits completo**
 - Directorio de **vector de bits asignado a grupos**
 - Directorio **limitado**
- Directorio **distribuido entre módulos y cachés**: además de distribuir las filas del directorio en MP, se distribuyen entre las cachés con copia del bloque.
 - Directorio **encadenado**

4.5 Conclusiones

Los protocolos de **snooping** presentan problemas de **escalabilidad** Una alternativa son los protocolos **basados en directorio**:

- En SMPs (Symmetric MultiProcessors): directorio centralizado
- Multicore (como Intel Core i7): directorio centralizado
- En DSMs (Distributed Shader Memory): directorio distribuido